In [1]:
```
import os
import numpy as np
import pandas as pd
import joblib

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.model_selection import GroupShuffleSplit

import xgboost as xgb
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import GRU, Dense, Dropout, Bidirectional, Layer, Inpu
from tensorflow.keras.callbacks import EarlyStopping
```

In [2]:
```
# Load data
data_path = "Data/NASA_CMAPSS/train_FD001.txt"

col_names = [
    'unit_number', 'time_in_cycles',
    'op1', 'op2', 'op3'
] + [f'sensor_{i}' for i in range(1, 22)]

df = pd.read_csv(data_path, sep='\s+', header=None)
df.columns = col_names
```

In [3]:
```
# Compute RUL
max_cycles = df.groupby('unit_number')['time_in_cycles'].max().reset_index()
max_cycles.columns = ['unit_number', 'max_cycle']

df = df.merge(max_cycles, on='unit_number')
df['RUL'] = df['max_cycle'] - df['time_in_cycles']

RUL_CAP = 125
df['RUL'] = df['RUL'].clip(upper=RUL_CAP)
```

In [4]:
```
# Normalize
sensor_cols = [c for c in df.columns if c.startswith('sensor_')]

scaler = StandardScaler()
df[sensor_cols] = scaler.fit_transform(df[sensor_cols])
```

In [5]:
```
# Create sequences
SEQ_LEN = 40

def create_sequences(df, sensor_cols, seq_len):
    X, y, groups = [], [], []

    for uid, g in df.groupby('unit_number'):
        g = g.sort_values('time_in_cycles').reset_index(drop=True)

        for i in range(len(g)):
```

```
            if i + 1 >= seq_len:
                seq = g.loc[i-seq_len+1:i, sensor_cols].values
                rul = g.loc[i, 'RUL']

                X.append(seq)
                y.append(rul)
                groups.append(uid)

    return np.array(X), np.array(y), np.array(groups)

X_seq, y_seq, groups = create_sequences(df, sensor_cols, SEQ_LEN)
```

In [6]:
```
# Attention Layer
class Attention(Layer):
    def __init__(self):
        super(Attention, self).__init__()

    def build(self, input_shape):
        self.W = self.add_weight(
            name="att_weight",
            shape=(input_shape[-1], 1),
            initializer="normal"
        )
        self.b = self.add_weight(
            name="att_bias",
            shape=(input_shape[1], 1),
            initializer="zeros"
        )
        super(Attention, self).build(input_shape)

    def call(self, x, return_attention=False):
        e = tf.keras.backend.tanh(tf.tensordot(x, self.W, axes=1) + self.b)
        a = tf.keras.backend.softmax(e, axis=1)

        output = x * a
        output = tf.keras.backend.sum(output, axis=1)

        if return_attention:
            return output, a
        return output
```

In [7]:
```
# Train-Test Split
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, test_idx = next(gss.split(X_seq, y_seq, groups))

X_train, X_test = X_seq[train_idx], X_seq[test_idx]
y_train, y_test = y_seq[train_idx], y_seq[test_idx]
```

In [8]:
```
# Building weights
# Stronger weight when RUL is small
def make_weights(y, alpha=2.0, cap=125.0):
    # weight ≈ 1 + alpha*(1 - y/cap)
    w = 1.0 + alpha * (1.0 - (y / cap))
    return w
```

```
w_train = make_weights(y_train, alpha=2.0)
```

In [9]:
```
# Train BiGRU
inputs = Input(shape=(SEQ_LEN, len(sensor_cols)))

x = Bidirectional(GRU(96, return_sequences=True))(inputs)
x = Dropout(0.3)(x)

x = GRU(48, return_sequences=True)(x)

att_layer = Attention()
x, att_weights = att_layer(x, return_attention=True)

x = Dense(64, activation='relu')(x)
outputs = Dense(1)(x)

model = Model(inputs, outputs)
attention_model = Model(inputs, att_weights)

model.compile(optimizer='adam', loss='mse', metrics=[], weighted_metrics=['mae'])

model.summary()

early_stop = EarlyStopping(monitor='val_loss', patience=7, restore_best_weights=Tru

model.fit(
    X_train, y_train,
    sample_weight=w_train,
    validation_split=0.1,
    epochs=50,
    batch_size=64,
    callbacks=[early_stop],
    verbose=1
)
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 40, 21)]          0

 bidirectional (Bidirection  (None, 40, 192)           68544
 al)

 dropout (Dropout)           (None, 40, 192)           0

 gru_1 (GRU)                 (None, 40, 48)            34848

 attention (Attention)       ((None, 48),              88
                              (None, 40, 1))

 dense (Dense)               (None, 64)                3136

 dense_1 (Dense)             (None, 1)                 65

=================================================================
Total params: 106681 (416.72 KB)
Trainable params: 106681 (416.72 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
Epoch 1/50
189/189 [==============================] - 22s 89ms/step - loss: 4112.8408 - mae: 3
6.2111 - val_loss: 969.6654 - val_mae: 18.8371
Epoch 2/50
189/189 [==============================] - 15s 80ms/step - loss: 640.4058 - mae: 14.
7176 - val_loss: 563.6342 - val_mae: 14.6836
Epoch 3/50
189/189 [==============================] - 15s 79ms/step - loss: 475.1829 - mae: 12.
3166 - val_loss: 299.1996 - val_mae: 9.8713
Epoch 4/50
189/189 [==============================] - 16s 85ms/step - loss: 319.6221 - mae: 9.8
147 - val_loss: 321.2080 - val_mae: 10.1314
Epoch 5/50
189/189 [==============================] - 16s 87ms/step - loss: 278.4542 - mae: 9.0
277 - val_loss: 275.8581 - val_mae: 8.3478
Epoch 6/50
189/189 [==============================] - 17s 90ms/step - loss: 249.5186 - mae: 8.5
178 - val_loss: 279.7850 - val_mae: 8.7009
Epoch 7/50
189/189 [==============================] - 16s 85ms/step - loss: 241.3812 - mae: 8.3
648 - val_loss: 260.3562 - val_mae: 8.8012
Epoch 8/50
189/189 [==============================] - 17s 89ms/step - loss: 224.1311 - mae: 8.0
227 - val_loss: 253.4721 - val_mae: 8.4342
Epoch 9/50
189/189 [==============================] - 17s 87ms/step - loss: 215.0785 - mae: 7.8
184 - val_loss: 263.0394 - val_mae: 8.8329
Epoch 10/50
189/189 [==============================] - 16s 86ms/step - loss: 206.5931 - mae: 7.6
777 - val_loss: 289.6562 - val_mae: 9.3166
Epoch 11/50
```

```
189/189 [==============================] - 17s 88ms/step - loss: 197.7347 - mae: 7.5
174 - val_loss: 307.3702 - val_mae: 9.0693
Epoch 12/50
189/189 [==============================] - 17s 89ms/step - loss: 189.3739 - mae: 7.2
970 - val_loss: 300.0337 - val_mae: 8.4306
Epoch 13/50
189/189 [==============================] - 18s 93ms/step - loss: 180.8556 - mae: 7.1
018 - val_loss: 280.6387 - val_mae: 8.5513
Epoch 14/50
189/189 [==============================] - 16s 87ms/step - loss: 168.5666 - mae: 6.9
519 - val_loss: 320.6404 - val_mae: 9.0465
Epoch 15/50
189/189 [==============================] - 16s 87ms/step - loss: 156.4521 - mae: 6.6
903 - val_loss: 326.3524 - val_mae: 9.0555
```

Out[9]: <keras.src.callbacks.History at 0x2206c1dd2d0>

In [10]:
```python
# Generate Residuals
train_pred = model.predict(X_train).ravel()
test_pred = model.predict(X_test).ravel()

residual_train = y_train - train_pred
```

```
421/421 [==============================] - 9s 18ms/step
103/103 [==============================] - 2s 19ms/step
```

In [48]:
```python
# Create XGBoost Features (strong statistical descriptors + BiGRU output)
def extract_features(X_seq):
    feats = []

    for seq in X_seq:
        f = []
        for i in range(seq.shape[1]):  # each sensor
            s = seq[:, i]

            # Basic stats
            mean = np.mean(s)
            std = np.std(s)
            min_val = np.min(s)
            max_val = np.max(s)
            last = s[-1]

            # Trend features
            slope = np.polyfit(range(len(s)), s, 1)[0]
            curvature = np.polyfit(range(len(s)), s, 2)[0]

            # New richer features
            median = np.median(s)
            p25 = np.percentile(s, 25)
            p75 = np.percentile(s, 75)
            value_range = max_val - min_val
            last_diff = s[-1] - s[-2] if len(s) > 1 else 0

            f.extend([
                mean, std, min_val, max_val, last,
                slope, curvature,
                median, p25, p75,
```

```
                value_range, last_diff
            ])

        feats.append(f)

    return np.array(feats)
```

In [72]:
```
X_train_feat = extract_features(X_train)
X_test_feat = extract_features(X_test)

# Add BiGRU prediction as feature
X_train_res = np.hstack([
    X_train_feat,
    train_pred.reshape(-1,1),
    (train_pred - np.mean(train_pred)).reshape(-1,1)  # bias signal
])
X_test_res = np.hstack([X_test_feat, test_pred.reshape(-1,1), (test_pred - np.mean(
```

In [83]:
```
import xgboost as xgb
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

# Train XGBoost on engineered features
xgb_baseline = xgb.XGBRegressor(
    n_estimators=500,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_lambda=1.0,
    random_state=42
)

xgb_baseline.fit(X_train_feat, y_train)

# Predictions
y_pred_xgb = xgb_baseline.predict(X_test_feat)
```

In [84]:
```
from sklearn.ensemble import RandomForestRegressor

rf_model = RandomForestRegressor(
    n_estimators=300,
    max_depth=12,
    min_samples_split=5,
    min_samples_leaf=2,
    n_jobs=-1,
    random_state=42
)

rf_model.fit(X_train_feat, y_train)

# Predictions
y_pred_rf = rf_model.predict(X_test_feat)
```

In [85]:
```
def safe_mape(y_true, y_pred):
    mask = y_true > 10
    return np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100

def evaluate_model(y_test, y_pred, name="Model"):
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    mape = safe_mape(y_test, y_pred)

    low_rul_mask = y_test < 30
    rmse_low = np.sqrt(mean_squared_error(
        y_test[low_rul_mask],
        y_pred[low_rul_mask]
    ))

    print(f"\n{name}")
    print(f"RMSE: {rmse:.4f}")
    print(f"MAE: {mae:.4f}")
    print(f"R2: {r2:.4f}")
    print(f"MAPE: {mape:.4f}")
    print(f"Low RUL RMSE: {rmse_low:.4f}")

    return rmse, mae, r2, mape, rmse_low
```

In [86]:
```
metrics_xgb = evaluate_model(y_test, y_pred_xgb, "XGBoost (Features Only)")
metrics_rf = evaluate_model(y_test, y_pred_rf, "Random Forest (Features Only)")
```

```
XGBoost (Features Only)
RMSE: 11.1061
MAE: 8.1480
R2: 0.9285
MAPE: 12.7043
Low RUL RMSE: 4.8886

Random Forest (Features Only)
RMSE: 12.6544
MAE: 9.0375
R2: 0.9072
MAPE: 13.7241
Low RUL RMSE: 4.5888
```

In [87]:
```
import pandas as pd

results = pd.DataFrame({
    "Metric": ["RMSE", "MAE", "R2", "MAPE", "Low RUL RMSE"],
    "XGBoost": metrics_xgb,
    "Random Forest": metrics_rf
})

print(results)
```

	Metric	XGBoost	Random Forest
0	RMSE	11.106103	12.654425
1	MAE	8.147961	9.037452
2	R2	0.928548	0.907237
3	MAPE	12.704324	13.724115
4	Low RUL RMSE	4.888604	4.588781

In [ ]:

In [73]:
```
# Train XGBoost on Residuals
xgb_model = xgb.XGBRegressor(
    n_estimators=400,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

xgb_model.fit(X_train_res, residual_train, sample_weight=w_train)
```

Out[73]:

XGBRegressor

▶ Parameters

In [74]:
```
from sklearn.feature_selection import SelectFromModel
# Select top features
selector = SelectFromModel(xgb_model, max_features=80, threshold=-np.inf)
selector.fit(X_train_res, residual_train)

X_train_sel = selector.transform(X_train_res)
X_test_sel = selector.transform(X_test_res)

xgb_model_fs = xgb.XGBRegressor(
    n_estimators=400,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

xgb_model_fs.fit(X_train_sel, residual_train, sample_weight=w_train)
residual_test_pred = xgb_model_fs.predict(X_test_sel)
```

In [96]:
```
feature_names = []

stat_names = [
    "mean", "std", "min", "max", "last",
    "slope", "curvature",
    "median", "p25", "p75",
    "range", "last_diff"
]
```

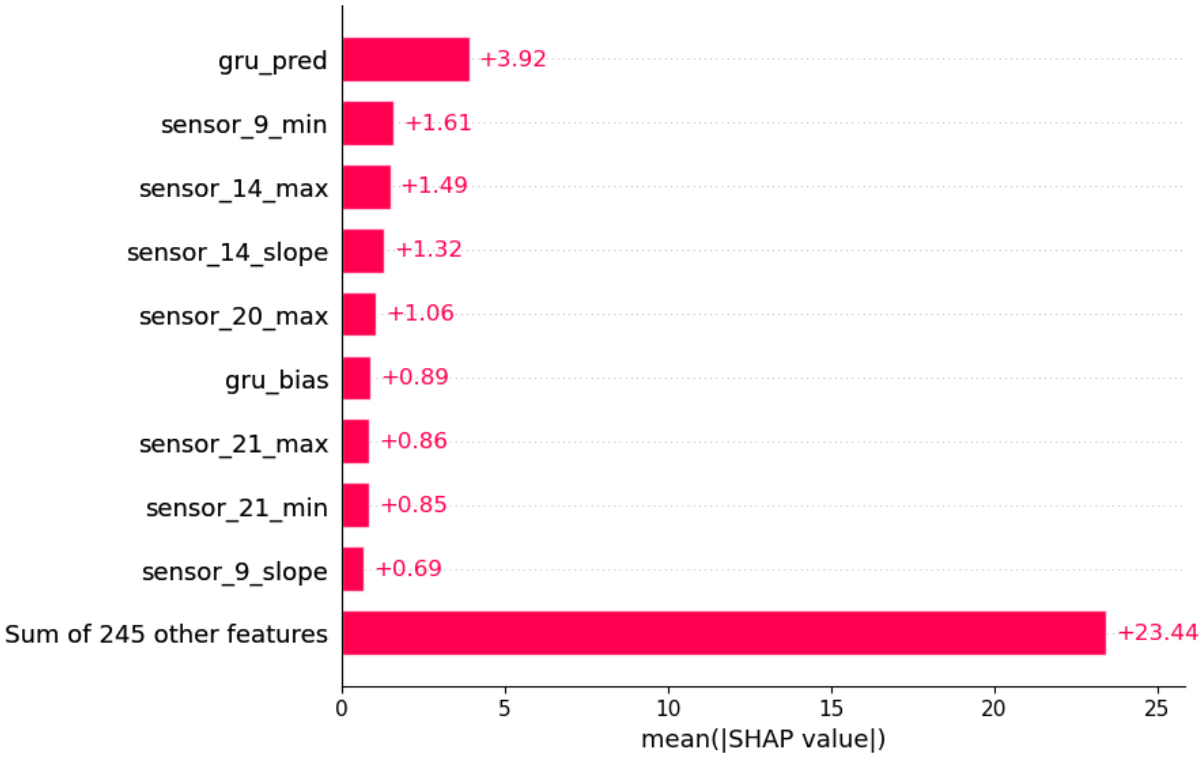
```
# Ensure ALL sensors are included
print("Sensors:", len(sensor_cols))  # should be 21

for sensor in sensor_cols:
    for stat in stat_names:
        feature_names.append(f"{sensor}_{stat}")

# Add GRU features
feature_names += ["gru_pred", "gru_bias"]

# Final check
print("Total features:", len(feature_names))
```

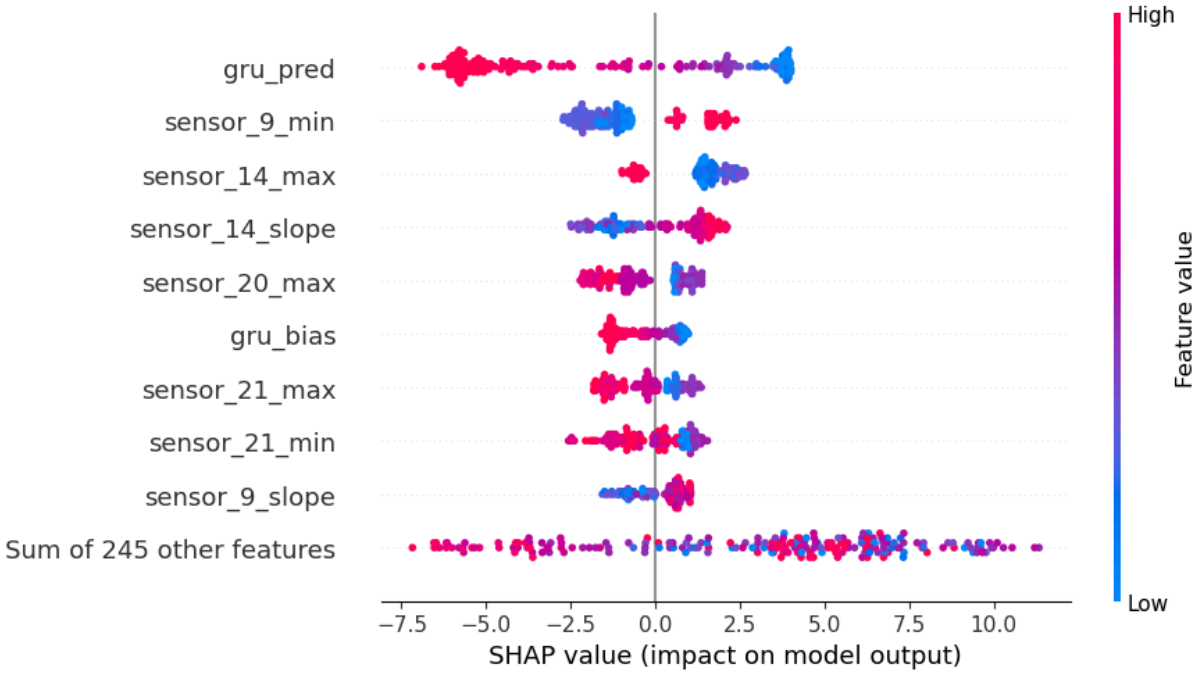
```
Sensors: 21
Total features: 254
```

In [97]:
```
print(X_train_res.shape[1])
print(len(feature_names))
```

```
254
254
```

In [98]:
```
import shap

# subset for speed
X_sample = X_test_res[:200]

explainer = shap.Explainer(xgb_model, X_train_res, feature_names=feature_names)
shap_values = explainer(X_sample)

# Global importance
shap.plots.bar(shap_values)

# Feature impact distribution
shap.plots.beeswarm(shap_values)
```





In [100…

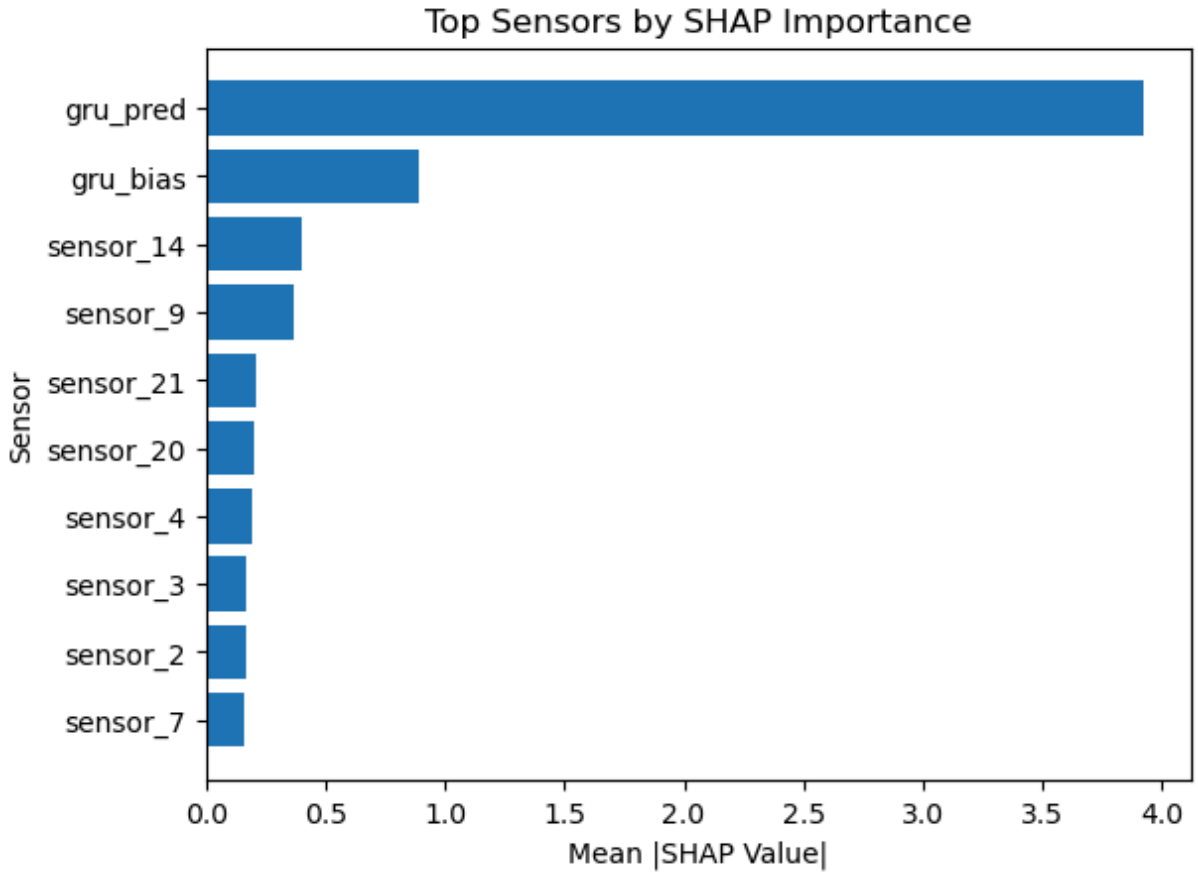
```
# Group SHAP by sensor
from collections import defaultdict
# SHAP values (samples × features)
shap_vals = shap_values.values   # shape: (n_samples, n_features)

# Sensor Mapping
sensor_map = []

for name in feature_names:
    if "sensor_" in name:
        sensor = "_".join(name.split("_")[:2])  # sensor_9_mean → sensor_9
    else:
```

```
        sensor = name  # gru_pred, gru_bias
    sensor_map.append(sensor)

sensor_shap = defaultdict(list)

# Collect SHAP values per sensor
for i, sensor in enumerate(sensor_map):
    sensor_shap[sensor].append(shap_vals[:, i])

# Aggregate (mean absolute SHAP)
sensor_importance = {}

for sensor, vals in sensor_shap.items():
    vals = np.stack(vals, axis=1)  # (samples, features_of_sensor)
    sensor_importance[sensor] = np.mean(np.abs(vals))
```

In [103…

```
# Rank Sensors Globally
# Sort sensors
sorted_sensors = sorted(
    sensor_importance.items(),
    key=lambda x: x[1],
    reverse=True
)

# Print top sensors
print("\nTop Sensors:")
for s, val in sorted_sensors[:10]:
    #if "gru" in s:
        #print("GRU Feature:", s, val)
    print(f"{s}: {val:.4f}")

# Plot Sensor-Level Importance

top_k = 10
top_sensors = sorted_sensors[:top_k]

names = [x[0] for x in top_sensors]
values = [x[1] for x in top_sensors]

plt.figure()
plt.barh(names[::-1], values[::-1])
plt.title("Top Sensors by SHAP Importance")
plt.xlabel("Mean |SHAP Value|")
plt.ylabel("Sensor")
plt.show()
```

```
Top Sensors:
gru_pred: 3.9238
gru_bias: 0.8881
sensor_14: 0.3990
sensor_9: 0.3706
sensor_21: 0.2083
sensor_20: 0.1977
sensor_4: 0.1941
sensor_3: 0.1717
sensor_2: 0.1645
sensor_7: 0.1636
```



In [63]:
```
residual_test_pred = xgb_model.predict(X_test_res)
```

In [76]:
```
# OLD residual prediction (no feature selection)
residual_test_pred_old = xgb_model.predict(X_test_res)

# Gamma tuning (OLD)
best_rmse_old = float("inf")
best_gamma_old = None
best_pred_old = None
gamma_values = np.linspace(0.3, 0.7, 21)

for g in gamma_values:
    hybrid_pred = test_pred + g * residual_test_pred_old
    rmse = np.sqrt(mean_squared_error(y_test, hybrid_pred))
    print(f"Gamma={g:.2f} | RMSE={rmse:.4f}")
    if rmse < best_rmse_old:
        best_rmse_old = rmse
```

```
        best_gamma_old = g
        best_pred_old = hybrid_pred

print("OLD Best Gamma:", best_gamma_old)
print("OLD RMSE:", best_rmse_old)
```

```
Gamma=0.30 | RMSE=10.9226
Gamma=0.32 | RMSE=10.9174
Gamma=0.34 | RMSE=10.9131
Gamma=0.36 | RMSE=10.9098
Gamma=0.38 | RMSE=10.9075
Gamma=0.40 | RMSE=10.9061
Gamma=0.42 | RMSE=10.9057
Gamma=0.44 | RMSE=10.9063
Gamma=0.46 | RMSE=10.9079
Gamma=0.48 | RMSE=10.9104
Gamma=0.50 | RMSE=10.9139
Gamma=0.52 | RMSE=10.9183
Gamma=0.54 | RMSE=10.9237
Gamma=0.56 | RMSE=10.9301
Gamma=0.58 | RMSE=10.9374
Gamma=0.60 | RMSE=10.9457
Gamma=0.62 | RMSE=10.9550
Gamma=0.64 | RMSE=10.9652
Gamma=0.66 | RMSE=10.9764
Gamma=0.68 | RMSE=10.9885
Gamma=0.70 | RMSE=11.0016
OLD Best Gamma: 0.42
OLD RMSE: 10.905745967377952
```

In [77]:
```
# NEW residual prediction (with feature selection)
residual_test_pred_new = xgb_model_fs.predict(X_test_sel)

# Gamma tuning (NEW)
best_rmse_new = float("inf")
best_gamma_new = None
best_pred_new = None
gamma_values = np.linspace(0.3, 0.7, 21)

for g in gamma_values:
    hybrid_pred = test_pred + g * residual_test_pred_new
    rmse = np.sqrt(mean_squared_error(y_test, hybrid_pred))

    if rmse < best_rmse_new:
        best_rmse_new = rmse
        best_gamma_new = g
        best_pred_new = hybrid_pred

print("NEW Best Gamma:", best_gamma_new)
print("NEW RMSE:", best_rmse_new)
```

```
NEW Best Gamma: 0.42
NEW RMSE: 10.888355341531476
```

In [78]:
```
import pandas as pd
from sklearn.metrics import r2_score
```

```python
def safe_mape(y_true, y_pred):
    mask = y_true > 10
    return np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100

def evaluate_model(y_test, y_pred, name="Model"):
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    mape = safe_mape(y_test, y_pred)

    low_rul_mask = y_test < 30
    rmse_low = np.sqrt(mean_squared_error(
        y_test[low_rul_mask],
        y_pred[low_rul_mask]
    ))

    print(f"\n{name}")
    print(f"RMSE: {rmse:.4f}")
    print(f"MAE:  {mae:.4f}")
    print(f"R2:   {r2:.4f}")
    print(f"MAPE: {mape:.4f}")
    print(f"Low RUL RMSE: {rmse_low:.4f}")

    return rmse, mae, r2, mape, rmse_low

# Evaluate OLD hybrid
metrics_old = evaluate_model(y_test, best_pred_old, "Hybrid (Before FS)")

# Evaluate NEW hybrid
metrics_new = evaluate_model(y_test, best_pred_new, "Hybrid (After FS)")

results = pd.DataFrame({
    "Metric": ["RMSE", "MAE", "R2", "MAPE", "Low RUL RMSE"],
    "Before FS": metrics_old,
    "After FS": metrics_new
})

print(results)
```

```
Hybrid (Before FS)
RMSE: 10.9057
MAE:  7.9012
R2:   0.9311
MAPE: 12.5663
Low RUL RMSE: 4.4321

Hybrid (After FS)
RMSE: 10.8884
MAE:  7.9056
R2:   0.9313
MAPE: 12.5206
Low RUL RMSE: 4.3837
         Metric  Before FS   After FS
0          RMSE  10.905746  10.888355
1           MAE   7.901174   7.905647
2            R2   0.931103   0.931322
3          MAPE  12.566336  12.520633
4  Low RUL RMSE   4.432109   4.383700
```

In [79]:

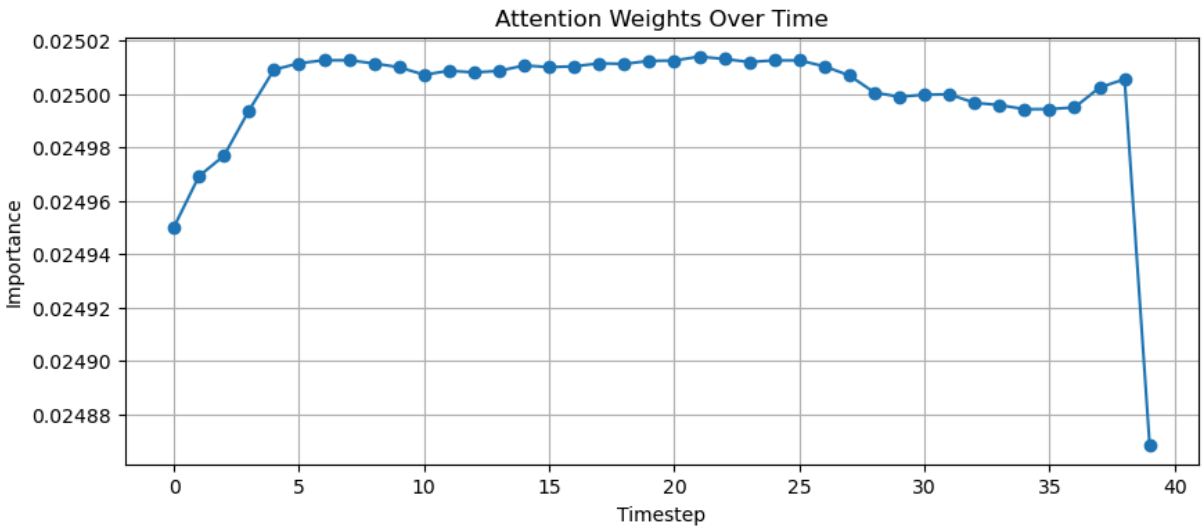
```python
from sklearn.metrics import r2_score

rmse_gru = np.sqrt(mean_squared_error(y_test, test_pred))
rmse_hybrid = best_rmse_old

mae_gru = mean_absolute_error(y_test, test_pred)
mae_hybrid = mean_absolute_error(y_test, best_pred_old)

r2_gru = r2_score(y_test, test_pred)
r2_hybrid = r2_score(y_test, best_pred_old)

# MAPE
def safe_mape(y_true, y_pred):
    mask = y_true > 10   # ignore near-zero RUL
    return np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100

mape_gru = safe_mape(y_test, test_pred)
mape_hybrid = safe_mape(y_test, best_pred_old)

low_rul_mask = y_test < 30

rmse_hybrid_low = np.sqrt(mean_squared_error(
    y_test[low_rul_mask],
    best_pred_old[low_rul_mask]
))
rmse_gru_low = np.sqrt(mean_squared_error(
    y_test[low_rul_mask],
    test_pred[low_rul_mask]
))

print("\nFINAL RESULTS")
print("BiGRU RMSE:", rmse_gru, rmse_gru_low)
print("Hybrid RMSE:", rmse_hybrid, rmse_hybrid_low)

print("\nBiGRU MAE:", mae_gru)
print("Hybrid MAE:", mae_hybrid)
```

```
print("\nBiGRU R2:", r2_gru)
print("Hybrid R2:", r2_hybrid)

print("\nBiGRU MAPE:", mape_gru)
print("Hybrid MAPE:", mape_hybrid)
```

```
FINAL RESULTS
BiGRU RMSE: 11.115495372034601 4.65201247771042
Hybrid RMSE: 10.905745967377952 4.432108653048429

BiGRU MAE: 7.920037269592285
Hybrid MAE: 7.901174068450928

BiGRU R2: 0.9284270405769348
Hybrid R2: 0.9311027526855469

BiGRU MAPE: 12.690954830592771
Hybrid MAPE: 12.56633634948755
```

In [80]:
```
import matplotlib.pyplot as plt
# One test sample
sample = X_test[0:1]

# Get attention weights
weights = attention_model.predict(sample)[0].squeeze()

plt.figure(figsize=(10,4))
plt.plot(weights, marker='o')
plt.title("Attention Weights Over Time")
plt.xlabel("Timestep")
plt.ylabel("Importance")
plt.grid()
plt.show()
```

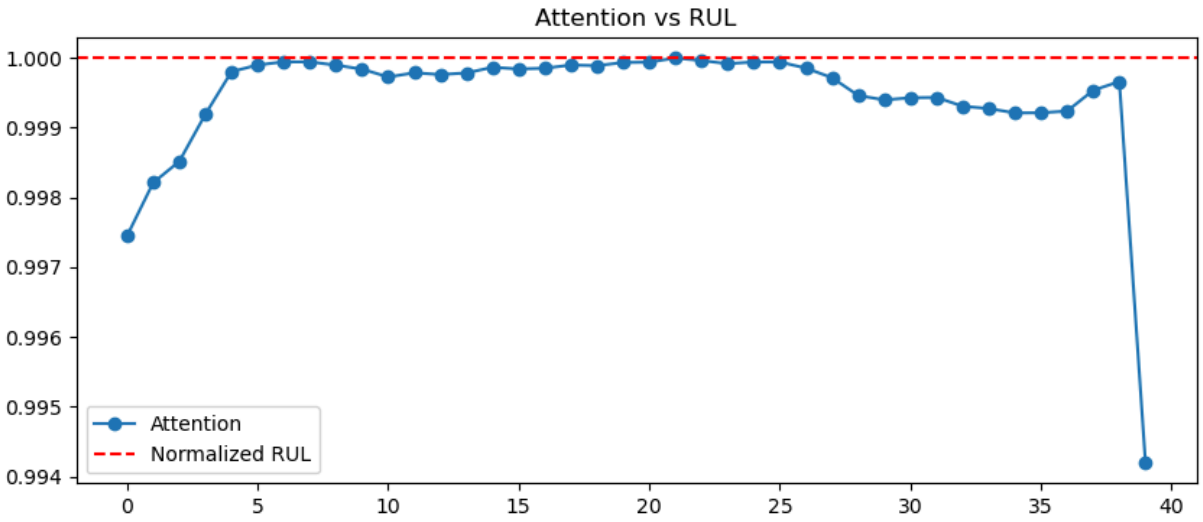
```
1/1 [==============================] - 0s 44ms/step
```



In [81]:
```
# Overlay with RUL
sample_idx = 0

weights = attention_model.predict(X_test[sample_idx:sample_idx+1])[0].squeeze()
```

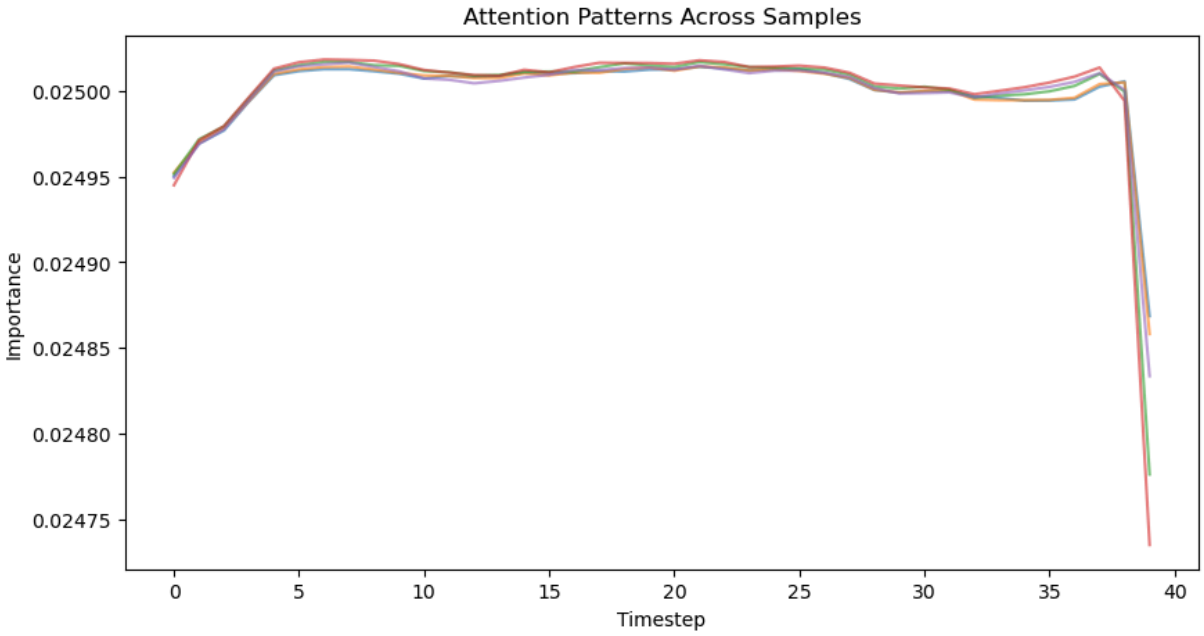
```
rul_seq = y_test[sample_idx]

plt.figure(figsize=(10,4))

plt.plot(weights / np.max(weights), label="Attention", marker='o')
plt.axhline(y=rul_seq/125, color='r', linestyle='--', label="Normalized RUL")

plt.legend()
plt.title("Attention vs RUL")
plt.show()
```

```
1/1 [==============================] - 0s 46ms/step
```

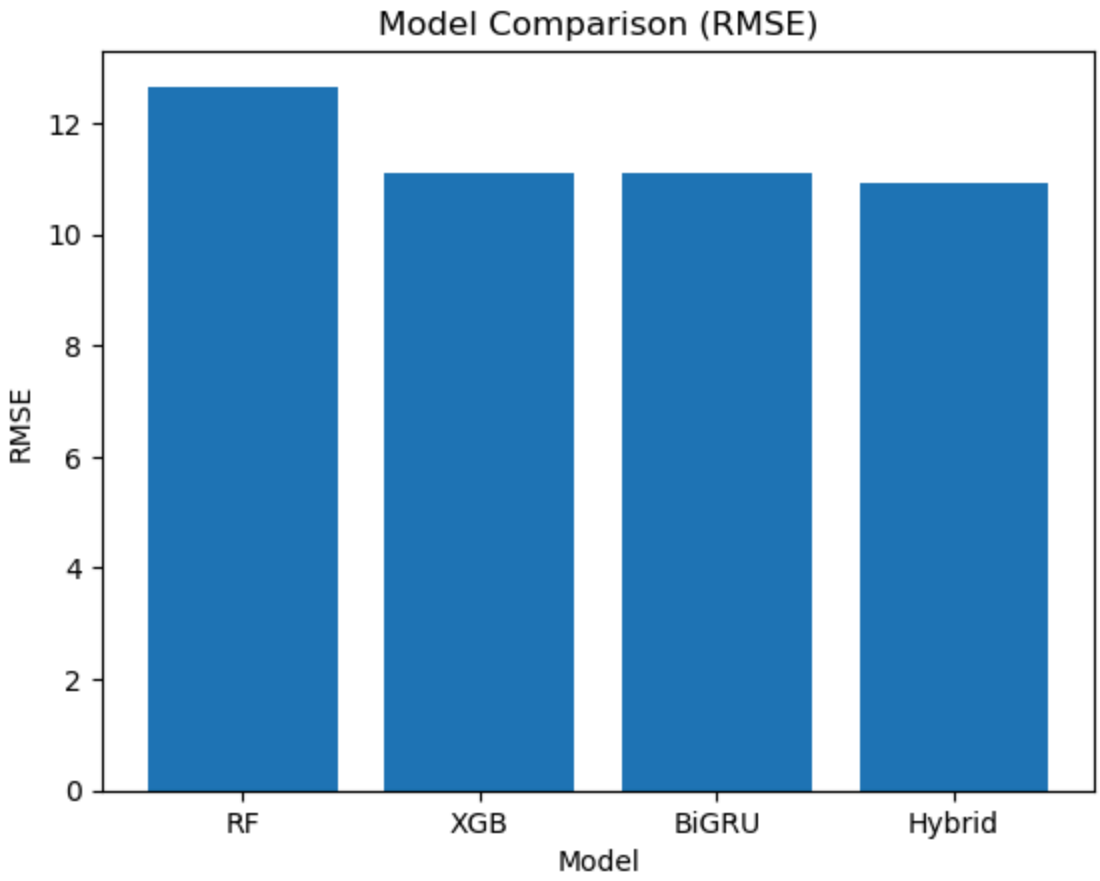


In [82]:
```
plt.figure(figsize=(10,5))

for i in range(5):
    weights = attention_model.predict(X_test[i:i+1])[0].squeeze()
    plt.plot(weights, alpha=0.6)

plt.title("Attention Patterns Across Samples")
plt.xlabel("Timestep")
plt.ylabel("Importance")
plt.show()
```

```
1/1 [==============================] - 0s 41ms/step
1/1 [==============================] - 0s 52ms/step
1/1 [==============================] - 0s 38ms/step
1/1 [==============================] - 0s 48ms/step
1/1 [==============================] - 0s 38ms/step
```



In [88]:
```
import matplotlib.pyplot as plt

models = ["RF", "XGB", "BiGRU", "Hybrid"]
rmse_values = [
    metrics_rf[0],
    metrics_xgb[0],
    rmse_gru,
    rmse_hybrid
]

plt.figure()
plt.bar(models, rmse_values)
plt.title("Model Comparison (RMSE)")
plt.xlabel("Model")
plt.ylabel("RMSE")
plt.show()
```



In [93]:
```
feature_names = []
stat_names = [
    "mean", "std", "min", "max", "last",
    "slope", "curvature",
    "median", "p25", "p75",
    "range", "last_diff"
]

for sensor in sensor_cols:
    for stat in stat_names:
        feature_names.append(f"{sensor}_{stat}")
```

```
Feature 97 → sensor_20_min
Feature 98 → sensor_20_max
Feature 101 → sensor_21_std
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[93], line 9
      7 for i in range(96, 253):
      8     if i == 101 or i == 97 or i==243 or i==98 or i==159 or i==242 or i==161
or i==231 or i == 252:
----> 9         print(f"Feature {i} → {feature_names[i]}")

IndexError: list index out of range
```

In [ ]: